Algomaniacs

Segment Trees

Shashwat Shukla
(shashwat1511)

Motivation

- Problem:

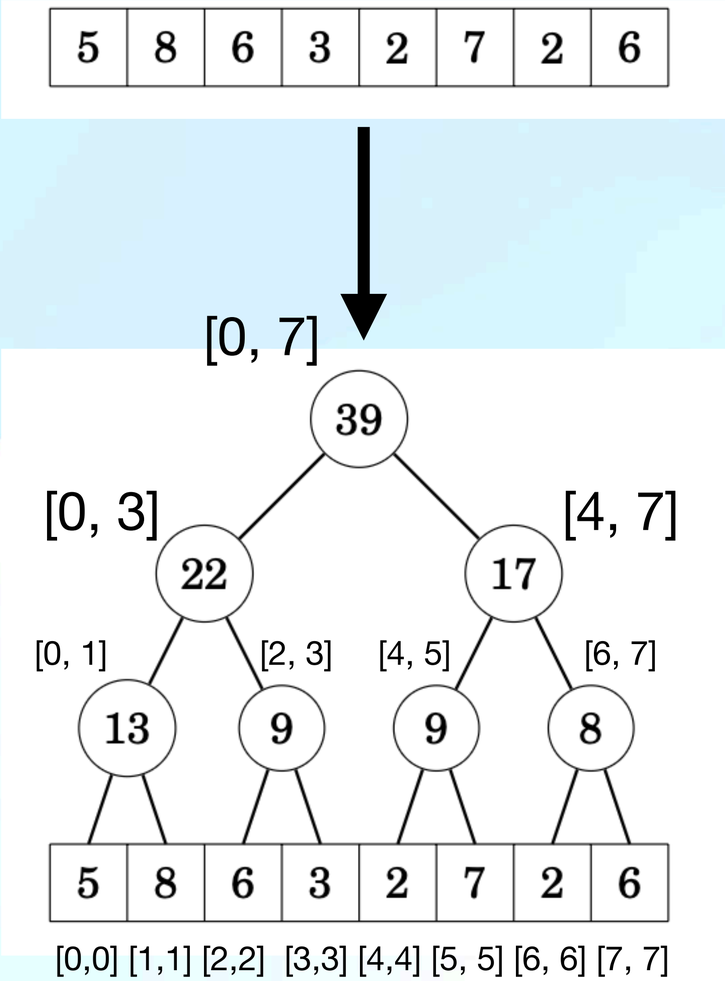
  Given an array of integers $a_1, a_2, \ldots, a_n$, perform 2 types of operations

  1. Update(i, x) -> change the value at index i to x

  2. Query(L, R) -> determine the sum of (a[L], a[L + 1], …, a[R]).

- Since the array(or the structure we are working on) is getting changed throughout the process, we can't rely on one time pre-computation alone(eg. sparse tables, prefix sums, etc).

Visual Representation of a Segment Tree



- Each node of the tree has either 0 or 2 children(Full Binary Tree), and this is true for any array size, it doesn't have to be a perfect power of 2.
- Each leaf node corresponds to the original array element.
- Each non-leaf node contains the sum of its children, which is basically a merge operation performed on its children.
- Each node contains the sum of a certain range in the array, so you can characterize each node by three variables - low, high, and sum.

Segment Tree Construction

```cpp
const int maxn = 1e5;

vector<int> tree(4 * maxn);

void construct(vector<int>& vec, int low, int high, int node)
{
    if(low > high)
        return;

    if(low == high)
    {
        tree[node] = vec[low];
        return;
    }

    int mid{(low + high)/2};
    construct(vec, low, mid, left_child(node));
    construct(vec, mid + 1, high, right_child(node));
    tree[node] = tree[left_child(node)] + tree[right_child(node)];
}
```

Using zero based indexing

```cpp
int left_child(int x)
{
    return 2 * x + 1;
}

int right_child(int x)
{
    return 2 * x + 2;
}
```

The time complexity is O(n)

The (point) update operation

```
void update(int low, int high, int node, int index, int new_val)
{
    if(index > high || index < low)
        return;

    if(index == low && index == high)
    {
        tree[node] = new_val;
        return;
    }

    int mid{(low + high)/2};
    update(low, mid, left_child(node), index, new_val);
    update(mid + 1, high, right_child(node), index, new_val);
    tree[node] = tree[left_child(node)] + tree[right_child(node)];
}
```

Range Query

```
int query(int query_l, int query_r, int low, int high, int node)
{
    if(query_l > high || query_r < low)
        return 0;

    if(query_l <= low && query_r >= high)
        return tree[node];

    int mid{(low + high)/2};
    int left{query(query_l, query_r, low, mid, left_child(node))};
    int right{query(query_l, query_r, mid + 1, high, right_child(node))};
    return left + right;
}
```

Why return 0 in particular when we are out of query range?

The Time Complexities

- If the cost of merging two nodes in a segment tree is denoted by $T_{merge} = O(d)$, then the total time complexity for an update or query operation T(n) is:

$$T(n) = O(d \cdot \log n)$$

- Here, the merge operation is simply an addition, therefore $T_{merge} = O(1)$, and the time complexity T(n) is simply logarithmic. How do we arrive at this tho?

- So, any range in the array can be covered completely by O(log n) nodes in our segment tree. All the recursive procedure is just an efficient method to find those nodes in O(log n) time.

Generalization

- We can represent each node of our tree as a structure(struct) in itself. Then the merge function takes in 2 nodes as arguments and returns a node.

- Now, some things that should be kept in mind while modelling a problem using a segment tree -

1. The merge function must be associative.

2. Identity Element I -> merge(a, I) = merge(I, a) = a
   0 in case of sum, -infinity in case of max, +infinity in case of min, etc. quite simple to figure out.

- Note that it isn't necessary for the merge function to be commutative, you just have to stay consistent with the order of your merge operations.

Generalization

```
struct Node   //change accordingly.
{
    int val;
    Node(int val)
    {
        this -> val = val;
    }
};

vector<Node> tree(4 * maxn);
Node NEUT{Node(0)};   //change accordingly.
```

```
Node merge(Node& left, Node& right)   //change accordingly.
{
    return Node(left.val + right.val);
}
```

Generalization

```
Node query(int query_l, int query_r, int low, int high, int node)
{
    if(query_l > high || query_r < low)
        return NEUT;

    if(query_l <= low && query_r >= high)
        return tree[node];

    int mid{(low + high)/2};
    Node left{query(query_l, query_r, low, mid, left_child(root))};
    Node right{query(query_l, query_r, mid + 1, high, right_child(root))};
    return merge(left, right);
}
```

- You can make a class for this whole data structure, which helps in modularizing the code even further. Highly recommended to write it yourself, instead of copy pasting others' templates.

Example

- Given an array of integers, for any range [l,r], print the minimum element in that range, and also the number of times it appears in the range.

- All we need to think of is what data we need to store in our node so that we can quickly merge two nodes, and what'll be our identity element.

- https://codeforces.com/edu/course/2/lesson/4/1/practice/contest/273169/problem/C -> Link to above problem

- https://codeforces.com/edu/course/2/lesson/4/2/practice/contest/273278/problem/A -> Maximum subarray sum queries

Reducing problems to known, easily solvable problems

- Given an array of integers A, an inversion is defined as a pair (i, j) such that $i < j$ and $A[i] > A[j]$. We define the inversion count of A as the no. of inversions in A.
- Problem Statement -> For a given array A of size n, find the minimum inversion count among all subarrays of A having length k.
- Constraints -> $1 \leq n \leq 1e5$, $1 \leq A[i] \leq n$, $1 \leq k \leq n$
- https://leetcode.com/problems/minimum-inversion-count-in-subarrays-of-fixed-length/description/

Using segment tree for optimizing dp transitions

- Given an array of integers A of length n, find the length of its longest increasing subsequence(can also ask to print the LIS but that's not relevant for this lecture).
- Constraints -> 1 ≤ n ≤ 1e5 , 1 ≤ A[i] ≤ n
- Note that there does exist a concise(although a bit non-trivial at first glance) greedy solution using binary search for this problem(having time complexity O(n logn)) as well.
- https://cses.fi/problemset/task/1145

```
int ans{1};
vector<int> dp(n, 1);
//dp[i] -> length of the LIS ending at index i
for(int i{}; i < n; ++i)
{
    for(int j{}; j < i; ++j)
    {
        if(A[j] < A[i])
            dp[i] = max(dp[i], 1 + dp[j]);
    }

    ans = max(ans, dp[i]);
}
```

Trivial dp solution. How do we optimize this?

Lazy Propagation

- Till now we were performing point updates on our array, and trying to answer queries of different types.
- Now, we'll be performing range updates, that is, we'll be performing some kind of operation on a particular array range. Most of the times that operation will be addition, but the idea can be generalized for different types of functions.
- We'll maintain a "lazy" array, which will store for each node, the total sum that needs to be added to the corresponding range.

Lazy Propagation

```
    void push(int low, int high, int node)
    {
        tree[node].val += lazy[node];
        if(low != high)
        {
            lazy[left_child(node)] += lazy[node];
            lazy[right_child(node)] += lazy[node];
        }

        lazy[node] = 0;
    }
```

The implementation is not hard at all.

All you need is to write a push/propagate function, and then add one line on top of your query function.

Note that your push function will change based on how the range operations are interacting with the data you are storing in your tree. For example, here we are storing min in the tree, so addition of x on any segment simply means that its min gets increased by x as well.

```
    Node query(int query_l, int query_r, int low, int high, int node)
    {
        if(lazy[node])
            push(low, high, node);

        if(query_l > high || query_r < low)
            return NEUT;

        if(query_l <= low && query_r >= high)
            return tree[node];

        int mid{(low + high)/2};
        Node left{query(query_l, query_r, low, mid, left_child(node))};
        Node right{query(query_l, query_r, mid + 1, high, right_child(node))};
        return merge(left, right);
    }
```

Lazy Propagation(The (range) update operation)

```
void update(int low, int high, int node, int query_l, int query_r, int to_add)
{
    if(lazy[node])
        push(low, high, node);

    if(query_l > high || query_r < low)
        return;

    if(query_l <= low && query_r >= high)
    {
        lazy[node] += to_add;
        push(low, high, node);
        return;
    }

    int mid{(low + high)/2};
    update(low, mid, left_child(node), query_l, query_r, to_add);
    update(mid + 1, high, right_child(node), query_l, query_r, to_add);
    tree[node] = merge(tree[left_child(node)], tree[right_child(node)]);
}
```

Problem

- https://codeforces.com/contest/2145/problem/E
- Most of the times you won’t have to come up with some crazy merge function for your segment tree, it will very often be min, max, gcd, some basic queries like that, for most problems rated <= 2100 on Codeforces. Modeling is the harder part.

More things to learn

- A lazy segment tree which allows you to perform assign(x) operation on a particular range(without the range add operations).
- More advanced -> A lazy segment tree that allows you to perform both the add(x) and assign(x) operations on ranges.
- Merge Sort Trees -> A very powerful data structure that can be used to perform queries like finding upper/lower bounds of given integer x on any array range in $O((\log n)^2)$ complexity, and much more. You can use binary search only on sorted arrays, this works for any arbitrary array.
- Persistent Segment Trees -> very rarely encountered if you are below rating 2100, I don't see a reason to study this apart from pure interest for most people(I myself haven't studied this yet lol).
- It's an endless list. I could keep writing but I've got to stop somewhere :)

Thank You!